

Ex.No: 1

Implementation and Time analysis of sorting algorithms.
Bubble sort, Selection sort, Insertion sort, Merge sort and
Quicksort

1. Bubble Sort

Objective

- To implement the Bubble Sort algorithm.
- To sort elements in ascending order by repeatedly swapping adjacent elements.
- To analyze its time complexity and performance.

code

```
#include <stdio.h>
```

```
void bubbleSort(int arr[], int n) {
```

```
    int i, j, temp;
```

```
    for(i = 0; i < n - 1; i++) {
```

```
        for(j = 0; j < n - i - 1; j++) {
```

```
            if(arr[j] > arr[j + 1]) {
```

```
                temp = arr[j];
```

```
                arr[j] = arr[j + 1];
```

```
                arr[j + 1] = temp;
```

```
            }
```

```
        }
```

```
    }
```

```
}  
  
void printArray(int arr[], int n) {  
    int i;  
    for(i = 0; i < n; i++)  
        printf("%d ", arr[i]);  
}  
  
int main() {  
    int arr[] = {64, 34, 25, 12, 22, 11, 90};  
    int n = sizeof(arr) / sizeof(arr[0]);  
  
    bubbleSort(arr, n);  
  
    printf("Sorted array: ");  
    printArray(arr, n);  
  
    return 0;  
}
```

Output

Sorted array: 11 12 22 25 34 64 90

=== Code Execution Successful ===

Analysis

- Bubble Sort compares adjacent elements and swaps them if they are in the wrong order.

- After each pass, the largest element moves to its correct position.
- It is simple to implement but inefficient for large datasets.
- Best suited for small or nearly sorted arrays.

Conclusion

Bubble Sort is easy to understand and implement. However, due to its **$O(n^2)$** complexity, it is not suitable for large datasets. It is mainly used for educational purposes.

2. Selection Sort

Objective

- To implement the Selection Sort algorithm.
- To sort elements by selecting the minimum element in each pass.
- To analyze its performance.

Code (C)

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, i, j, min, temp;
```

```
    printf("Enter number of elements: ");
```

```
    scanf("%d", &n);
```

```
    int a[n];
```

```
    printf("Enter elements:\n");
```

```
    for(i = 0; i < n; i++)
```

```
        scanf("%d", &a[i]);
```

```
    for(i = 0; i < n - 1; i++) {
```

```
        min = i;
```

```
        for(j = i + 1; j < n; j++) {
```

```
            if(a[j] < a[min])
```

```
                min = j;
```

```
        }
```

```
temp = a[i];  
a[i] = a[min];  
a[min] = temp;  
}  
  
printf("Sorted array:\n");  
for(i = 0; i < n; i++)  
    printf("%d ", a[i]);  
  
return 0;  
}
```

Out put: Enter number of elements: 5

Enter elements:

64

34

25

12

22

Sorted array:

12 22 25 34 64

Analysis

- Finds the minimum element in the unsorted portion.
- Swaps it with the first unsorted element.

Conclusion

Selection Sort is easy to implement and performs fewer swaps, but it is inefficient for large datasets.

4. Merge Sort

Objective

- To implement the Merge Sort algorithm.
- To sort elements using the Divide and Conquer technique.

- To analyze its performance.

Code (C)

```
#include <stdio.h>
```

```
void merge(int a[], int l, int m, int r) {
```

```
    int i, j, k;
```

```
    int n1 = m - l + 1;
```

```
    int n2 = r - m;
```

```
    int L[n1], R[n2];
```

```
    for(i = 0; i < n1; i++)
```

```
        L[i] = a[l + i];
```

```
    for(j = 0; j < n2; j++)
```

```
        R[j] = a[m + 1 + j];
```

```
    i = j = 0;
```

```
    k = l;
```

```
    while(i < n1 && j < n2) {
```

```
        if(L[i] <= R[j])
```

```
            a[k++] = L[i++];
```

```
        else

            a[k++] = R[j++];

    }

    while(i < n1)

        a[k++] = L[i++];

    while(j < n2)

        a[k++] = R[j++];

    }

void mergeSort(int a[], int l, int r) {

    if(l < r) {

        int m = (l + r) / 2;

        mergeSort(a, l, m);

        mergeSort(a, m + 1, r);

        merge(a, l, m, r);

    }

}

int main() {

    int n, i;

    printf("Enter number of elements: ");
```

```
scanf("%d", &n);

int a[n];

printf("Enter elements:\n");

for(i = 0; i < n; i++)

    scanf("%d", &a[i]);

mergeSort(a, 0, n - 1);

printf("Sorted array:\n");

for(i = 0; i < n; i++)

    printf("%d ", a[i]);

return 0;

}
```

Output

Enter number of elements: 5

Enter elements:

64

34

25

12

22

Sorted array:

12 22 25 34 64

Analysis

- Merge Sort divides the array into smaller subarrays and merges them after sorting.
- It guarantees $O(n \log n)$ performance in all cases.

Conclusion

Merge Sort is highly efficient for large datasets but requires extra memory during merging.

5. Quick Sort

Objective

- To implement the Quick Sort algorithm.
- To sort elements using partitioning.
- To analyze its efficiency.

Code (C)

```
#include <stdio.h>
```

```
int partition(int a[], int low, int high) {
```

```
    int pivot = a[high];
```

```
    int i = low - 1, temp;
```

```
    for(int j = low; j < high; j++) {
```

```
        if(a[j] < pivot) {
```

```
            i++;
```

```
            temp = a[i];
```

```
            a[i] = a[j];
```

```
            a[j] = temp;
```

```
        }
```

```
    }
```

```
    temp = a[i + 1];
```

```
    a[i + 1] = a[high];
```

```
    a[high] = temp;
```

```
    return i + 1;
```

```
}
```

```
void quickSort(int a[], int low, int high) {
```

```
    if(low < high) {
```

```
        int pi = partition(a, low, high);

        quickSort(a, low, pi - 1);

        quickSort(a, pi + 1, high);

    }

}
```

```
int main() {

    int n, i;


    printf("Enter number of elements: ");

    scanf("%d", &n);


    int a[n];


    printf("Enter elements:\n");

    for(i = 0; i < n; i++)

        scanf("%d", &a[i]);


    quickSort(a, 0, n - 1);


    printf("Sorted array:\n");

    for(i = 0; i < n; i++)

        printf("%d ", a[i]);
```

```
    return 0;  
}
```

Output

Enter number of elements: 5

Enter elements:

64

34

25

12

22

Sorted array:

12 22 25 34 64

Analysis

- Quick Sort partitions the array around a pivot element.
- It is one of the fastest sorting algorithms in practice.

Time Complexity

- Best Case: $O(n \log n)$
- Average Case: $O(n \log n)$
- Worst Case: $O(n^2)$

Conclusion

Quick Sort provides excellent average-case performance and is widely used for sorting large datasets. Its worst case occurs only when poor pivot selection leads to highly unbalanced partitions.



Marwadi
University
Marwadi Chandarana Group



FACULTY OF TECHNOLOGY
Department of CE – AI ML DS
01AI0404 – Computer Network